QFT y estimación de fase

Genesis Silesky

21 de octubre de 2026

1. Transformada de Fourier cuántica y estimación de fase

1.1. Objetivo

Construir la transformada de Fourier cuántica (QFT) con un circuito y usarla para estimar la fase de un autovalor (estimación de fase, QPE).

1.2. Conceptos previos

Concepto previo: fases, puertas controladas y números binarios

Ver capítulos 05a (fases e interferencia) y 14 (puertas controladas). Una fase se puede escribir como una fracción binaria: $0.\varphi_1\varphi_2\ldots\varphi_m = \varphi_1/2 + \varphi_2/4 + \cdots + \varphi_m/2^m$. Por ejemplo $0.75 = 0.11_2$ y $1/4 = 0.01_2$.

1.3. Fundamento teórico

QFT. La QFT es un componente central de muchos algoritmos (Shor y la estimación de fase, entre ellos). Es la versión cuántica de la transformada de Fourier discreta (DFT). Con $N = 2^n$ estados de la base, lleva

$$|j\rangle \longmapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i\, jk/N} |k\rangle\,, \tag{1}$$

una matriz unitaria con entradas $U[k,j] = e^{2\pi ijk/N}/\sqrt{N}$. Aunque la fórmula se parece a la de la DFT, el significado es distinto: la DFT transforma una función discreta en el dominio del tiempo y la QFT transforma un estado cuántico en otro.

El circuito. Escribiendo $j = j_1 j_2 \ldots j_n$ en binario, la QFT se puede reescribir como un producto de qubits:

$$|j\rangle \mapsto \frac{1}{2^{n/2}} \big(\,|0\rangle + e^{2\pi i\, 0.j_n}\,|1\rangle\,\big)\big(\,|0\rangle + e^{2\pi i\, 0.j_{n-1}j_n}\,|1\rangle\,\big)\cdots\big(\,|0\rangle + e^{2\pi i\, 0.j_1 j_2 \ldots j_n}\,|1\rangle\,\big). \tag{2}$$

Esto da un circuito con tres tipos de pieza:

1. una Hadamard en cada qubit;
2. fases controladas CR_k, que aplican la fase $e^{2\pi i/2^k}$ a $|1\rangle$ (CR_2 es la puerta S controlada) y trasladan la información de los demás bits a la fase;
3. al final, puertas SWAP que invierten el orden de los qubits.

El número de puertas crece como $O(n^2)$, es decir, $O((\log N)^2)$ para N estados.

QPE. Dada una unitaria U y un autovector $|\psi\rangle$ con $U|\psi\rangle = e^{2\pi i\varphi}|\psi\rangle$, la estimación de fase calcula φ. La QFT pasa información del estado a la fase; la QPE pasa información de la fase al estado. Usa dos registros: el primero con m qubits (da una precisión de m bits) y el segundo, inicializado en el autovector $|\psi\rangle$. Pasos:

1. Primer registro en $|0\rangle^{\otimes m}$ y segundo en $|\psi\rangle$.
2. Hadamard en cada qubit del primer registro.
3. Puertas U^{2^k} controladas por el qubit k del primer registro. Cada una deja una fase $e^{2\pi i\varphi 2^k}$ en $|1\rangle$.
4. QFT inversa sobre el primer registro.
5. Se mide el primer registro: sale el número binario $\varphi_1\varphi_2\ldots\varphi_m$.

Por ejemplo, la puerta S tiene autovector $|1\rangle$ con autovalor $i = e^{2\pi i\cdot 1/4}$, así que $\varphi = 1/4 = 0.01_2$.

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 17 - Transformada de Fourier cuantica (QFT) y estimacion de fase (QPE)

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Operator, Statevector

sim = AerSimulator()

# %% Celda 2: la QFT como matriz
# La QFT lleva cada estado de la base |j> al estado
#     |j>  ->  (1/sqrt(N)) * suma_k  e^{2 pi i j k / N} |k>,     N = 2^n.
# Es la version cuantica de la transformada de Fourier discreta (DFT) y es una
# matriz unitaria con entradas  U[k, j] = e^{2 pi i j k / N} / sqrt(N).
def matriz_qft(n):
    N = 2 ** n
    j, k = np.meshgrid(np.arange(N), np.arange(N))
    return np.exp(2j * np.pi * j * k / N) / np.sqrt(N)


U = matriz_qft(3)
print("La QFT es unitaria (U^dagger U = I):", np.allclose(U.conj().T @ U, np.eye(8)))


# %% Celda 3: el circuito de la QFT
# Para cada qubit, de arriba hacia abajo:
#   1. Una Hadamard.
#   2. Fases controladas CR_k por cada qubit siguiente, que pasan la informacion
#      de los demas bits a la fase de |1>. CR_k aplica la fase e^{2 pi i / 2^k}
#      (CR_2 es la puerta S controlada).
# Al final una serie de SWAP invierte el orden de los qubits.
# Qiskit numera el qubit 0 a la derecha, asi que el bit mas significativo es el
# qubit n-1 y se empieza por el.
def circuito_qft(n):
    qc = QuantumCircuit(n)
    for objetivo in reversed(range(n)):
        qc.h(objetivo)
        for control in range(objetivo - 1, -1, -1):
            k = objetivo - control + 1
            qc.cp(2 * np.pi / 2 ** k, control, objetivo)
```

```
    for i in range(n // 2):
        qc.swap(i, n - 1 - i)
    return qc


for n in (2, 3, 4):
    coincide = np.allclose(Operator(circuito_qft(n)).data, matriz_qft(n))
    puertas = sum(circuito_qft(n).count_ops().values())
    print(f"n = {n}: el circuito coincide con la matriz de la QFT: {coincide} ({puertas} puertas
    ↪ )")

# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
circuito_qft(3).draw()

# %% Celda 4: la QFT de dos qubits sobre cada estado de la base
# Cada entrada |j> da cuatro amplitudes iguales en modulo (1/2) y lo que cambia es la
# fase: e^{2 pi i j k / 4} para cada k, es decir, multiplos de 90 grados.
qft2 = circuito_qft(2)
print("Fases (en grados) de las amplitudes de |k> para k = 00, 01, 10, 11:")
for j in range(4):
    entrada = format(j, "02b")
    amplitudes = Statevector.from_label(entrada).evolve(qft2).data
    # Los indices de Statevector crecen con el valor binario de la cadena (q1 q0).
    fases = [round(float(np.degrees(np.angle(c))) % 360) for c in amplitudes]
    print(f"  |{entrada}> -> {fases}")

# %% Celda 5: estimacion de fase (QPE)
# Sea U una puerta unitaria y |psi> un autovector con U|psi> = e^{2 pi i phi}|psi>.
# QPE estima phi con m bits: phi = 0.phi1 phi2 ... phim (en binario).
#   1. Primer registro (m qubits) en |0>, segundo registro en el autovector |psi>.
#   2. Hadamard en cada qubit del primer registro.
#   3. Puertas U^(2^k) controladas por el qubit k del primer registro.
#   4. QFT inversa sobre el primer registro.
#   5. Se mide el primer registro: sale el numero binario phi1 ... phim.
# La QFT pasa informacion de la base a la fase; la QPE hace lo contrario.
def qpe(phi, m):
    qc = QuantumCircuit(m + 1, m)
    qc.x(m)                       # el qubit m es el segundo registro: autovector |1> de la
    ↪ puerta fase
    qc.h(range(m))
    for k in range(m):
        # La puerta de fase P(2 pi phi) tiene autovalor e^{2 pi i phi} en |1>.
        # Controlada por el qubit k y elevada a 2^k: fase 2 pi phi 2^k.
        qc.cp(2 * np.pi * phi * 2 ** k, k, m)
    qc.compose(circuito_qft(m).inverse(), qubits=range(m), inplace=True)
    qc.measure(range(m), range(m))
    return qc


def estimar(phi, m, shots=1000):
    cuentas = sim.run(qpe(phi, m), shots=shots).result().get_counts()
    mejor = max(cuentas, key=cuentas.get)
    return mejor, int(mejor, 2) / 2 ** m, cuentas[mejor] / shots


# %% Celda 6: fases que se pueden escribir exactamente con m bits
# La puerta S tiene autovector |1> con autovalor i = e^{2 pi i (1/4)}, o sea phi = 1/4.
# En binario 1/4 = 0.01, y con m = 2 la QPE deberia dar 01.
# La puerta T tiene phi = 1/8 = 0.001: hacen falta m = 3.
for nombre, phi, m in (("S", 1 / 4, 2), ("T", 1 / 8, 3), ("Z", 1 / 2, 2)):
    bits, estimada, prob = estimar(phi, m)
    print(f"puerta {nombre}: phi = {phi:.3f} | QPE con m = {m}: {bits} -> {estimada:.3f} (
    ↪ probabilidad {prob:.2f})")
```

```python

# %% Celda 7: fases que NO caben exactamente
# phi = 1/3 no se puede escribir con pocos bits. La QPE da el valor mas cercano con
# alta probabilidad, y mas bits dan mas precision.
phi = 1 / 3
for m in (2, 4, 6):
    bits, estimada, prob = estimar(phi, m, shots=2000)
    print(f"m = {m}: resultado mas probable {bits} -> {estimada:.4f} | error = {abs(estimada - phi):.4f} | probabilidad {prob:.2f}")
```

1.5. Explicación celda por celda

Celda 2 Construye la matriz de la QFT y comprueba que es unitaria.

Celda 3 Construye el circuito (Hadamard, fases controladas y SWAP) y comprueba que su matriz es igual a la de la QFT para $n = 2, 3, 4$.

Celda 4 Aplica la QFT de dos qubits a cada estado de la base y muestra las fases de las cuatro amplitudes.

Celda 5 Define la QPE: Hadamard, fases controladas con $2\pi\varphi 2^k$, QFT inversa y medición.

Celda 6 Prueba fases exactas: puertas S (1/4), T (1/8) y Z (1/2).

Celda 7 Prueba $\varphi = 1/3$, que no se escribe exactamente con pocos bits, con $m = 2, 4, 6$.

1.6. Salida obtenida

```
La QFT es unitaria (U^dagger U = I): True
n = 2: el circuito coincide con la matriz de la QFT: True (4 puertas)
n = 3: el circuito coincide con la matriz de la QFT: True (7 puertas)
n = 4: el circuito coincide con la matriz de la QFT: True (12 puertas)
Fases (en grados) de las amplitudes de |k> para k = 00, 01, 10, 11:
  |00> -> [0, 0, 0, 0]
  |01> -> [0, 90, 180, 270]
  |10> -> [0, 180, 0, 180]
  |11> -> [0, 270, 180, 90]
puerta S: phi = 0.250 | QPE con m = 2: 01 -> 0.250 (probabilidad 1.00)
puerta T: phi = 0.125 | QPE con m = 3: 001 -> 0.125 (probabilidad 1.00)
puerta Z: phi = 0.500 | QPE con m = 2: 10 -> 0.500 (probabilidad 1.00)
m = 2: resultado mas probable 01 -> 0.2500 | error = 0.0833 | probabilidad 0.70
m = 4: resultado mas probable 0101 -> 0.3125 | error = 0.0208 | probabilidad 0.69
m = 6: resultado mas probable 010101 -> 0.3281 | error = 0.0052 | probabilidad 0.70
```

(El dibujo del circuito no se reproduce aquí.) Las cuentas de la QPE dependen del azar de las mediciones; el resultado más probable es estable.

1.7. Interpretación

El circuito coincide con la matriz de la QFT. Con 2 qubits, cada entrada da cuatro amplitudes del mismo módulo y fases que son múltiplos de 90°; por ejemplo $|01\rangle \to [0, 90, 180, 270]$. Con las fases exactas, la QPE da el resultado correcto con probabilidad 1: $S \to 01$ (0.25), $T \to 001$ (0.125) y $Z \to 10$ (0.5). Para $\varphi = 1/3$, el resultado más probable se acerca cada vez más: el error baja de 0.083 ($m = 2$) a 0.021 ($m = 4$) y a 0.005 ($m = 6$), con probabilidad cercana a 0.69.

1.8. Ejercicios

1. Comprueba la QPE con $\varphi = 3/8$ y $m = 3$. ¿Qué cadena esperas?
2. ¿Qué cadena da la QPE para la puerta Z con $m = 3$ bits? Predícelo y compruébalo.
3. Cuenta las puertas del circuito de la QFT para $n = 5$ y compara con n^2.

1.9. Errores comunes

- Olvidar los SWAP finales: sin ellos el circuito da la QFT con los qubits en orden inverso.
- Esperar un resultado exacto cuando la fase no cabe en m bits: la QPE da el valor más cercano, con alta probabilidad pero no exacta.